

MOVE-AI

기능 구현 원리 & 발표 스토리 가이드

“기사의 경험을 지식으로 남기고, 다음 배송에서 다시 쓰는 서비스”

핵심 한 줄

현장 암묵지 → 구조화 지식 →

상황별 Last 100m 안내

1

음성 제보

기사가 실제 말투로 팁을 남김

2

AI 구조화

한 문장을 여러 개의 원자 지식으로 분리

3

사람 검수

AI 결과를 관리자 승인 후에만 사용

4

RAG 재사용

승인 지식을 현재 장소·차량·구간에 맞춰 검색

왜 MOVE-AI가 필요한가?

지도에는 주소가 있지만, 실제 배송에 필요한 “현장 경험”은 기사 개인에게 흩어져 있습니다.

BEFORE

기존 배송

“어디에 차를 대야 하지?”

“큰 탑차도 들어가나?”

“화물 엘리베이터가 어디지?”

→ 같은 장소를 웹만 반복 학습

AFTER

MOVE-AI

베테랑의 경험을 음성으로 축적

AI가 검색 가능한 구조로 정리

관리자 승인으로 신뢰성 확보

→ 개인 경험에 기반한 서비스 제공한 데이터로

한 기사의 제보가 다음 기사의 안내가 되기까지

PPT에서는 아래 흐름을 한 장으로 보여주면 서비스 구조가 가장 빠르게 이해됩니다.



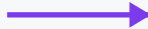
핵심: AI가 길을 만드는 것이 아니라, 사람이 만든 Route에 검증된 현장 지식을 붙여주는 구조

“문장을 예쁘게 요약”하는 게 아니라, 검색 가능한 원자 지식으로 분리

AI 출력은 곧바로 사용하지 않고 Draft로 저장한 뒤 관리자가 승인합니다.

기사 원문

“1.5톤 탑차는 정문이 좀 좁고요,
후문으로 들어가서 뒤에 세우면 돼요.
점심에는 엘리베이터가 오래 걸려요.”



K1 정문 1.5톤 탑차 진입 어려움
차량 / 조건부

K2 후문 후문으로 차량 진입
차량 / 행동안내

K3 엘리베이터 점심시간 대기 증가
시간조건 / 경고

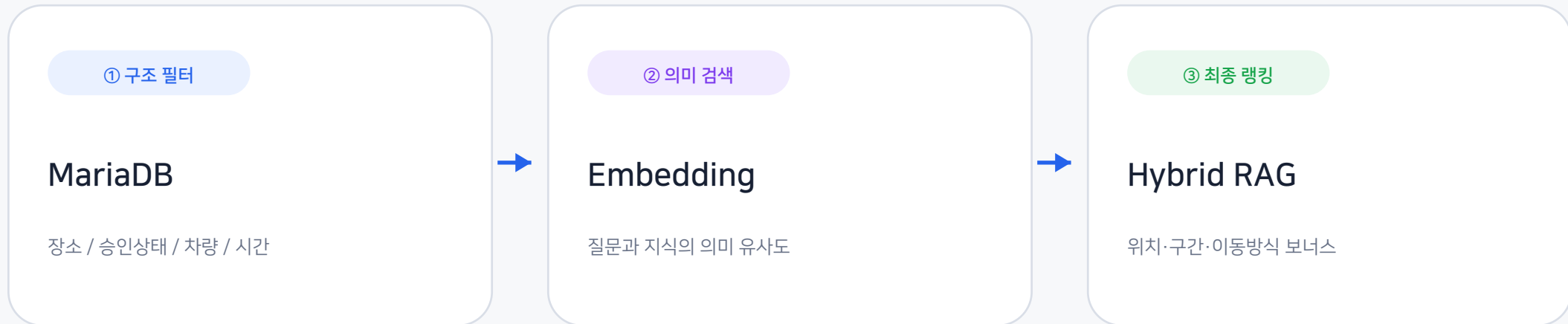
AI가 하면 안 되는 것

- × 원문에 없는 제한 숫자 만들기
- × 모르는 위치를 비슷한 Node에 억지 연결

- × 없는 대체 경로 만들기
- × 관리자 승인 없이 바로 RAG에 사용

왜 “벡터 검색”과 “구조 조건”을 같이 쓰는가?

현재 지식 규모에서는 별도 Vector DB 없이 MariaDB에 벡터를 저장하고, Python에서 cosine similarity를 계산합니다.



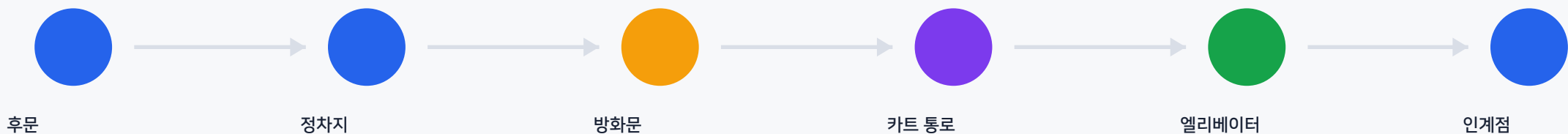
예시

질문: “5톤인데 어느 게이트로 들어가?”

벡터 유사도만 쓰면 1톤/5톤 지식이 둘 다 비슷하게 나올 수 있음 → 톤수 조건은 코드로 먼저/함께 판정

Route는 고정, RAG는 “현재 단계에 맞는 팁”을 붙인다

디자이너 화면에서는 “지도/경로 자체”보다 각 단계에 붙는 현장 팁이 차별점으로 보여야 합니다.



STEP 3 / 5

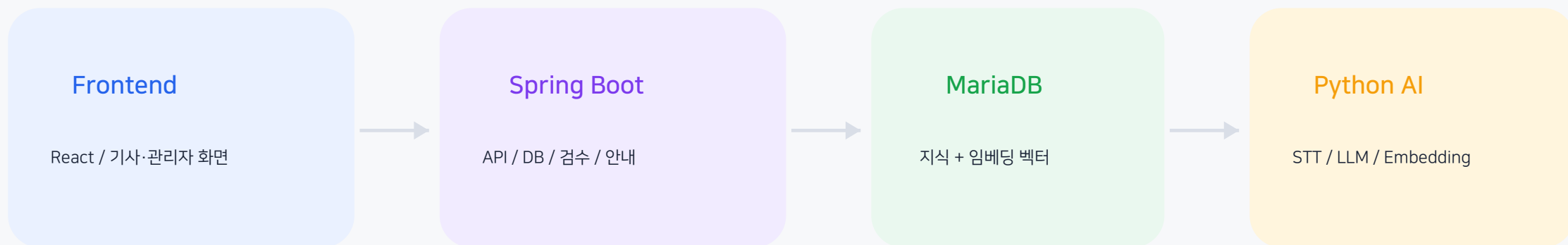
방화문 → 카트 통로

문턱에 카트 바퀴 걸림 주의

→ 속도를 줄여 천천히 통과

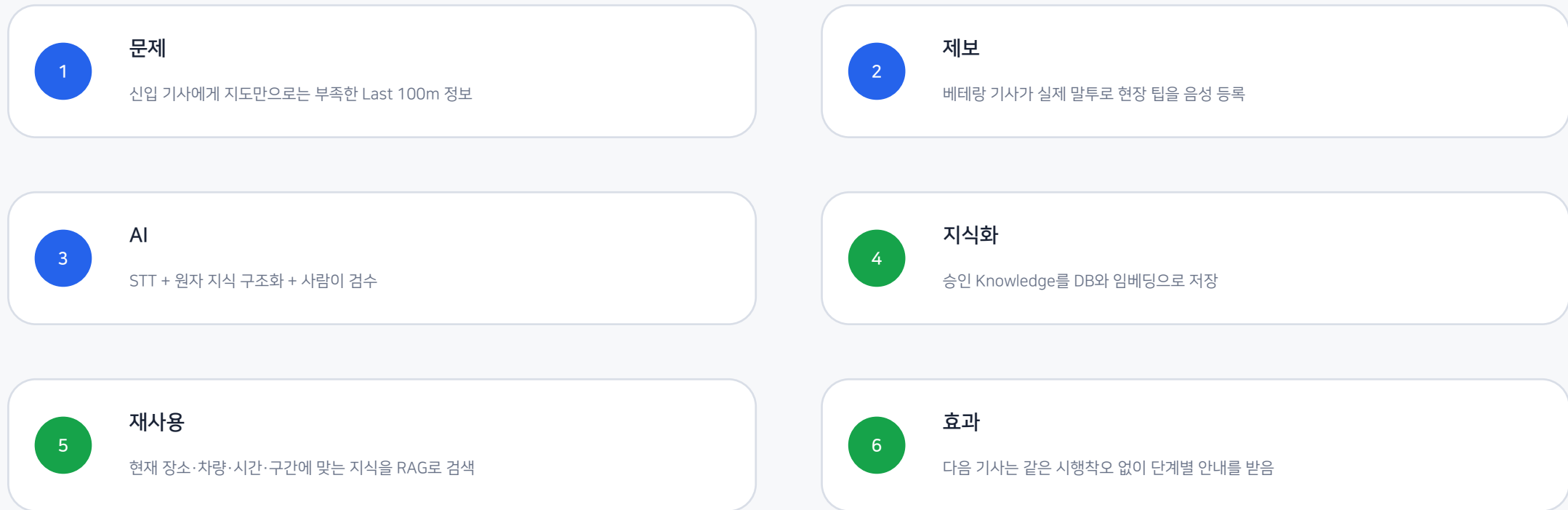
기획/디자인이 알아야 할 “어디까지 실제 구현되는가”

본선에서는 전부 완벽히 자동화하는 것보다 아래 PO 흐름이 실제로 연결되는 것이 중요합니다.



기능 설명보다 “경험이 어떻게 다음 배송에 재사용되는지”를 스토리로

아래 6장 정도를 중심으로 PPT를 잡으면 기술 설명과 사용자 가치가 자연스럽게 연결됩니다.



디자인 키워드

“경로 위에 지식” · “AI 자동결정이 아니라 사람 검수” · “Before/After” · “새 지식이 다음 배송에 반영”